Heidelberg University
Hardware and Artificial Intelligence (HAWAII) Lab

Holger Fröning
Arjan Siddhpura and Gina Rachor

High-Performance and Distributed Computing
Summer 2026

Exercise 3

- **Hand in via Moodle until 09:00 on Wednesday 20 May, 2026**
- **Include all names on the top sheet. Hand in a single PDF.**
- **Compress your results into a single archive (`.zip` or `.tar.gz`).**
- **Employ the following naming convention `hpc<XX>_ex<N>.{zip,tar.gz}`, where `XX` is your group number, and `N` is the number of the current exercise.**
- **A maximum of three students is allowed to collaborate on the exercises.**
- **In case an exercise requires programming:**
  - **include clean and documented code**
  - **include a Makefile for compiling**

3.1 Matrix multiply - parallel version using MPI

- The goal is to develop a parallel program performing a matrix multiply operation of the form $C = A \cdot B$. Matrices can be assumed to be square.
- Start with your sequential version from the previous exercise. It is up to you to use the non-optimized or optimized version. Implement an MPI-based parallel version of this program. Ensure the following:
  - Dynamic allocation of all matrices
  - Verify results of parallel execution with sequential execution
- Measure execution time, but do not include initialization. The initialization has to be done solely by one master rank which then distributes the input data. The broadcast of A and B is *not* a part of the initialization.
- You do not need to distribute C, it can be assumed to be 0 initially.
- Initialize the matrices according to: $A[i,j] = i + j, B[i,j] = i \cdot j$
- Execute with two (worker) ranks and report C for an input of 5x5.

(20 points)

C[i,j]	C[i,0]	C[i,1]	C[i,2]	C[i,3]	C[i,4]
C[0,j]					
C[1,j]					
C[2,j]					
C[3,j]					

3.2 Matrix multiply - scaling ranks

- Execute on the cluster and report the execution times, operating on 2048x2048 matrices for the following configurations:

# Nodes	World Sizes (distributed evenly on all nodes)
1	[02, 04, 08, 12, 16, 20, 24]
2	[02, 04, 12, 24, 36, 48]
4	[04, 12, 24, 48, 72, 96]

- Compare with your sequential (optimized) implementation. Report speed-up and efficiency and interpret these results. Add graphics where helpful for improved interpretation, trends are often much more visible this way.

(30 points)

World Size	# Nodes	Time [s]	Speed-Up	Efficiency
Sequential	1		1.0	100%
02	1			
04	1			
08	1			
12	1			
16	1			
20	1			
24	1			
02	2			
04	2			
12	2			
24	2			
36	2			
04	4			
12	4			
24	4			
48	4			
72	4			
96	4			

3.3 Matrix multiply - scaling problem size

- Execute on the cluster with 12 and 24 ranks, distribtued evenly on two nodes. Vary the problem size from 128x128 to 16384x16384 (all powers of two), calculate resulting GFLOP/s and interpret your results! Add graphics where helpful for improved interpretation, trends are often much more visible this way.

(20 points)

Problem Size	FLOPS	Time [s]	GFLOP/s
128			
256			
512			
1024			
2048			
4096			
8192			

3.4 Willingness to present

Please declare whether you are willing to present any of the previous exercises.
The declaration can be made on a per-exercise basis. Each declaration corresponds to 50% of the exercise points. You can only declare your willingness to present exercises for which you also hand in a solution. If no willingness to present is declared you may still be required to present an exercise for which your group has handed in a solution. This may happen if as example nobody else has declared their willingness to present.

- Matrix multiply - parallel version using MPI (Section: 3.1)
- Matrix multiply - scaling ranks (Section: 3.2)
- Matrix multiply - scaling problem size (Section: 3.3)

(10+15+10 points)

Total: 70+35 points